

## INTRODUCCIÓN A LA PROGRAMACIÓN ORIENTADA A OBJETOS

### Clases abstractas

Dr. Luciano H. Tamargo  
<http://cs.uns.edu.ar/~lt>  
Depto. de Ciencias e Ingeniería de la Computación  
Universidad Nacional del Sur, Bahía Blanca

## Programación Orientada a Objetos

Abstracción de datos

Encapsulamiento

Herencia

Polimorfismo

Ligadura dinámica

## CASO DE ESTUDIO: GESTIÓN RESIDUOS

- En un sistema de control de residuos un **terrero** se representa a través de una matriz de  $n \times n$
- Cada elemento de la matriz *puede* mantener un **contenedor** de tipo **papelero** o resistente para mantener residuos **contaminantes**.
- Todo contenedor tiene una **densidad** y un **volumen** y provoca un **impacto** sobre el medio ambiente.
- Un papelerero tiene una cantidad de fijaciones. El impacto ambiental de un papelerero es igual a su volumen dividido por dos.
- Un contenedor contaminante tiene un nivel de **toxicidad** que incide en el impacto ambiental, el impacto es el producto del volumen por la densidad por la toxicidad.

## CASO DE ESTUDIO: GESTIÓN RESIDUOS

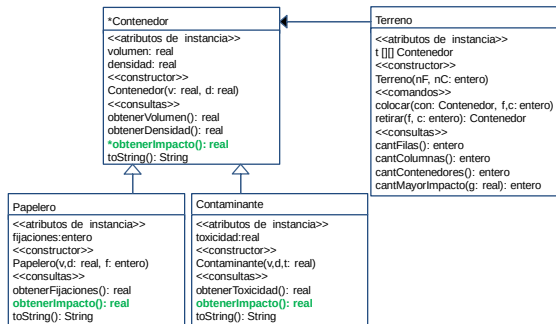


## CLASES ABSTRACTAS

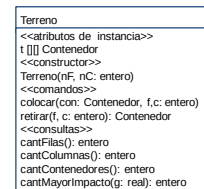
- En el diseño de una aplicación es posible definir una clase que factoriza propiedades de otras clases más específicas, **sin que existan en el problema objetos concretos** vinculados a esta clase más general.
- En este caso la clase se dice **abstracta** porque fue creada artificialmente para lograr un diseño que modele la realidad.
- En ejecución **no va a haber objetos** de software de una clase abstracta.

## CLASES ABSTRACTAS

- Una clase abstracta puede incluir uno, varios, todos o ningún método abstracto.
- Si una clase hereda de una clase abstracta y no implementa todos los métodos abstractos, también debe ser definida como abstracta.
- Una clase concreta debe implementar todos los métodos abstractos de sus clases ancestro.
- El constructor de una clase abstracta sólo va a ser invocado desde los constructores de las clases derivadas.
- Una clase abstracta puede tener atributos.



- **Terreno(nF,nC:entero)**  
Requiere nF y nC mayores a 0
- **colocar (con:Contenedor,f,c:entero)**  
Requiere el parámetro con ligado
- **retirar(f,c:entero):Contenedor**  
Asigna nulo a la posición f,c del terreno y retorna el contenedor que estaba asignado previamente
- **cantFilas():entero**  
Retorna la cantidad de filas
- **cantColumnas():entero**  
Retorna la cantidad de columnas
- **cantContenedores():entero**  
Retorna la cantidad de posiciones ligadas a contenedores
- **cantMayorImpacto(g:real):entero**  
Retorna la cantidad de posiciones ligadas a contenedores con impacto mayor a g



## CASO DE ESTUDIO: GESTIÓN RESIDUOS

```

abstract class Contenedor{
    //atributos de instancia
    protected float volumen;
    protected float densidad;
    //constructor
    public Contenedor(float v,float d){
        volumen = v; densidad = d;}
    //consultas
    public float obtenerVolumen(){
        return volumen;}
    public float obtenerDensidad(){
        return densidad;}
    abstract public float obtenerImpacto();
}
    
```

## CASO DE ESTUDIO: GESTIÓN RESIDUOS

```

class Contaminante extends Contenedor{
    //atributos de instancia
    protected float toxicidad;
    //constructor
    public Contaminante(float v, float d, float t){
        super(v,d);
        toxicidad = t; }
    //consultas
    public float obtenerToxicidad(){
        return toxicidad;}
    public float obtenerImpacto(){
        return volumen * densidad * toxicidad; }}
    
```

La consulta `obtenerImpacto` en la clase `Contaminante` redefine a la consulta definida en la clase `Contenedor`.

## CASO DE ESTUDIO: GESTIÓN RESIDUOS

```

class Papelero extends Contenedor{
    //atributos de instancia
    protected int fijaciones;
    //constructor
    public Papelero(float v, float d, int f){
        super(v,d);
        fijaciones = f; }
    //consultas
    public float obtenerFijaciones(){
        return fijaciones;}
    public float obtenerImpacto(){
        return volumen / 2; }}
    
```

La consulta `obtenerImpacto` en la clase `Papelero` redefine a la consulta definida en la clase `Contenedor`.

```

class Contenedor{
    public String toString(){
        return volumen + " " + densidad;
    }
}
    
```

```

class Contaminante extends Contenedor{
    public String toString(){
        return super.toString() + " " + toxicidad;
    }
}
    
```

```

class Papelero extends Contenedor{
    public String toString(){
        return super.toString() + " " + fijaciones;
    }
}
    
```

## CASO DE ESTUDIO: GESTIÓN RESIDUOS

```
class Terreno{
    //atributos de instancia
    private Contenedor [][]t ;
    //constructor
    public Terreno(int nF, int nC){
        t = new Contenedor[nF][nC];
    }
    ...
}
```

- El arreglo `t` es una **estructura de datos polimórfica**, los elementos pueden ser referencias a objetos de la clase **Contaminante** o de la clase **Papelero**.

## CASO DE ESTUDIO: GESTIÓN RESIDUOS

```
//comandos
public void colocar(Contenedor con, int f, int c){
    //Requiere la posición válida y con ligado
    t[f][c] = con;
}

public Contenedor retirar(int f, int c){
    //Requiere la posición válida
    Contenedor con = null;
    con = t[f][c];
    t[f][c] = null;
    return con;
}
```

- El comando `colocar` es un **método polimórfico**, recibe como parámetro a una **variable polimórfica**.

## CASO DE ESTUDIO: GESTIÓN RESIDUOS

```
//consultas
public boolean hayContenedor(int f, int c){
    //Requiere la posición válida
    return t[f][c] != null;
}
```

## CASO DE ESTUDIO: GESTIÓN RESIDUOS

```
public int cantMayorImpacto(float g){
    /*Retorna la cantidad de posiciones del terreno ligadas a
    contenedores con impacto mayor a g*/
    int cant = 0;
    for (int i = 0; i < cantFilas(); i++)
        for (int j = 0; j < cantColumnas(); j++)
            if (t[i][j] != null)
                if (t[i][j].obtenerImpacto() > g)
                    cant++;
    return cant;
}
```

- La ligadura entre el mensaje y el método `obtenerImpacto` es **dinámica**, se resuelve en ejecución y depende de la clase del objeto que recibe el mensaje.

## CASO DE ESTUDIO: GESTIÓN RESIDUOS

```
Terreno
<<atributos de instancia>>
t [][] Contenedor
<<constructor>>
Terreno(nF, nC: entero)
<<comandos>>
colocar(c: Contenedor, f, c: entero)
retirar(f, c: entero): Contenedor
<<consultas>>
hayContenedor(f, c: entero): boolean
impactoTerreno(): real
mayorToxicidad(): real
```

Computa la mayor toxicidad entre los contenedores contaminantes. Si no se registra toxicidad retorna 0.

## CASO DE ESTUDIO: GESTIÓN RESIDUOS

```
//consultas
public float mayorToxicidad(){
    float mayor = 0;
    for (int i = 0; i < cantFilas(); i++)
        for (int j = 0; j < cantColumnas(); j++)
            if (t[i][j] != null && t[i][j].obtenerToxicidad() > mayor)
                mayor = t[i][j].obtenerToxicidad();
    return mayor;
}
```

**ERROR**  
en compilación

- El **compilador reporta un error** porque no puede asegurar que `t[i][j]` pueda recibir el mensaje `obtenerToxicidad`.

## CASO DE ESTUDIO: GESTIÓN RESIDUOS

```
//consultas
public float mayorToxicidad(){
    float mayor = 0; Contaminante c;
    for (int i = 0; i < cantFilas(); i++)
        for (int j = 0; j < cantColumnas(); j++)
            if (t[i][j] != null)
                if (t[i][j] instanceof Contaminante) {
                    c = (Contaminante) t[i][j];
                    if (c.obtenerToxicidad() > mayor)
                        mayor = c.obtenerToxicidad();
                }
    return mayor;
}
```

La solución es correcta pero **no corresponde** a un diseño orientado a objetos.

## CASO DE ESTUDIO: GESTIÓN RESIDUOS

```
if (t[i][j] instanceof Contaminante)
```

- Computa **true** si `t[i][j]` mantiene una referencia a un objeto de clase `Contaminante`.

```
c = (Contaminante) t[i][j];
```

- La asignación es válida porque el **casting** relaja el control del compilador.

```
mayor = c.obtenerToxicidad();
```

- El mensaje es válido porque el tipo estático de `c` es `Contaminante`.

La solución es correcta pero **no corresponde** a un diseño orientado a objetos.

## CASO DE ESTUDIO: GESTIÓN RESIDUOS

```
//consultas
public float mayorToxicidad(){
    float mayor = 0; Contaminante c;
    for (int i = 0; i < cantFilas(); i++)
        for (int j = 0; j < cantColumnas(); j++)
            if (t[i][j] != null) {
                c = (Contaminante) t[i][j];
                if (c.obtenerToxicidad() > mayor)
                    mayor = c.obtenerToxicidad();
            }
    return mayor;
}
```

**ERROR**  
en ejecución

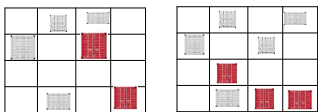
- El compilador no reporta error pero **la ejecución terminará anormalmente**, si la tabla contiene referencias a contenedores que no son contaminantes.

## CASO DE ESTUDIO: GESTIÓN RESIDUOS

```
Terreno
<<atributos de instancia>>
t [][] Contenedor
<<constructor>>
Terreno(n: entero)
<<comandos>>
colocar(c: Contenedor, f, c: entero)
retirar(f, c: entero): Contenedor
<<consultas>>
hayContenedor(f, c: entero): boolean
impactoTerreno(): real
mayorToxicidad(): real
filasMellizas(otro: Terreno): boolean
```

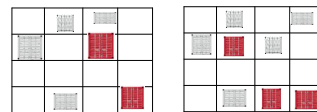
Requiere *otro* ligado. Computa true si existe `i` ( $0 \leq i \leq \text{cantFilas}()$ ) tal que la `i`ésima fila del terreno que recibe el mensaje y la `i`ésima fila de `otro` tienen todos los contenedores dispuestos en exactamente las mismas posiciones o ninguno de los dos tienen contenedores en esa fila.

## CASO DE ESTUDIO: GESTIÓN RESIDUOS



- La segunda fila hace que se verifique la propiedad `filasMellizas`.

## CASO DE ESTUDIO: GESTIÓN RESIDUOS



- La tercera fila hace que se verifique la propiedad `filasMellizas`.

## CASO DE ESTUDIO: GESTIÓN RESIDUOS

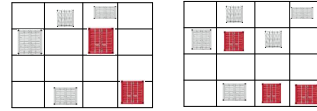


- No se verifica la propiedad

```
01100
10011
10010
01110
01100
10011
10011
10110
01110
10011
0111
0 0
0 0
0 0
```

25

## CASO DE ESTUDIO: GESTIÓN RESIDUOS



- Como el diseñador no expresó explícitamente que hacer si no tienen el mismo número de filas y columnas, consideramos que para que se cumpla la propiedad los terrenos deberían tener el mismo número de filas y columnas y lo agregamos como comentario en el código.
- Como programadores podríamos haber asumido otra cosa, por ejemplo si una matriz tiene 4 filas y otra 5, buscar si la propiedad se cumplen en 4 filas.

```
01100
10011
10010
01110
01100
10011
10011
10110
01110
10011
0111
0 0
0 0
0 0
```

26

## CASO DE ESTUDIO: GESTIÓN RESIDUOS

### Algoritmo filasMellizas

Si los terrenos tienen el mismo número de filas y columnas  
para cada fila y mientras no se encuentren dos filas mellizas  
para cada columna y mientras las filas puedan ser mellizas  
avanzar en la columna  
avanzar en la fila

```
01100
10011
10010
01110
01100
10011
10011
10110
01110
10011
0111
0 0
0 0
0 0
```

27

## CASO DE ESTUDIO: GESTIÓN RESIDUOS

```
//consultas
public boolean filasMellizas(Terreno otro){
    boolean mellizas = false;
    if (cantFilas() == otro.cantFilas() &&
        cantColumnas() == otro.cantColumnas()){
        for (int i = 0; i < cantFilas() && !mellizas; i++){
            mellizas = true;
            for (int j = 0; j < cantColumnas() && mellizas; j++){
                mellizas = (t[i][j]==null &&
                    !otro.hayContenedor(i,j)) ||
                    (t[i][j]!=null &&
                     otro.hayContenedor(i,j));
            }
        }
        return mellizas;
    }
}
```

- Observemos que `t` es un arreglo de dos dimensiones, la variable `otro` mantiene una referencia a un objeto de clase `Terreno`.

```
0
1
0
0
0
0
0
0
1
0
0
```